# Sign Me Up: Decoding Sign Language with Autoencoders

**Anirudh Sanjay Mhaske**
Department of Computer Science & Engineering
University at Buffalo
Buffalo, New York, USA
amhaske2@buffalo.edu

**Suvidha Vidyasagar Deshpande**
Department of Computer Science & Engineering
University at Buffalo
Buffalo, New York, USA
suvidhav@buffalo.edu

**Vidit Rajesh Prabhu**
Department of Computer Science & Engineering
University at Buffalo
Buffalo, New York, USA
viditraj@buffalo.edu

***Abstract*— This project uses Autoencoders to decode and reconstruct sign language gestures from images, enabling better communication for individuals with hearing or speech impairments. It also detects anomalies using reconstruction error. The model is designed to be robust against noise and provides latent space visualizations for deeper analysis.**

***Keywords*— *Sign Language, Autoencoders, Gesture Recognition, Anomaly Detection, Deep Learning, Denoising, Latent Space Visualization***

## I. Introduction

For individuals with hearing and speech impairments, sign language serves as a primary means of communication. However, its limited understanding among the general public often leads to communication barriers. This project focuses on creating a deep learning solution that can recognize and interpret sign language gestures from images. Using Autoencoders, the model learns to capture key features—such as hand shape and orientation—and reconstruct them accurately. The system also includes anomaly detection capabilities to identify irregular or noisy inputs, improving reliability and making it suitable for real-world applications.

## II. Problem Statement

Sign language is a critical communication tool for individuals with hearing and speech impairments, but its limited understanding among the general population creates a communication gap. Traditional gesture recognition systems often lack the accuracy and robustness required for real-world scenarios, especially when dealing with noisy or unfamiliar inputs. This project aims to develop an Autoencoder-based model that can effectively extract and reconstruct key features from sign language images, while also identifying anomalous gestures using reconstruction error, ensuring both accurate recognition and reliability.

## III. Background

Communication barriers still exist for individuals with hearing or speech impairments, primarily due to the limited understanding of sign language among the general population. Advances in deep learning, particularly in image-based models, have made it possible to develop intelligent systems capable of interpreting visual patterns like hand gestures. Autoencoders are well-suited for this task, offering the ability to extract key features, reconstruct images, and detect irregularities. This project leverages these strengths to design a model that can accurately interpret and reconstruct sign gestures while identifying anomalies, promoting more inclusive communication.

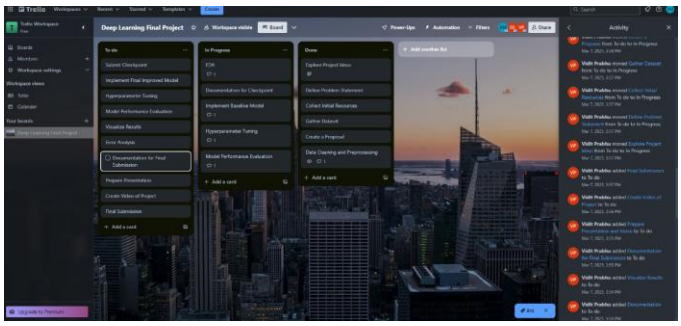
## IV. Methodology

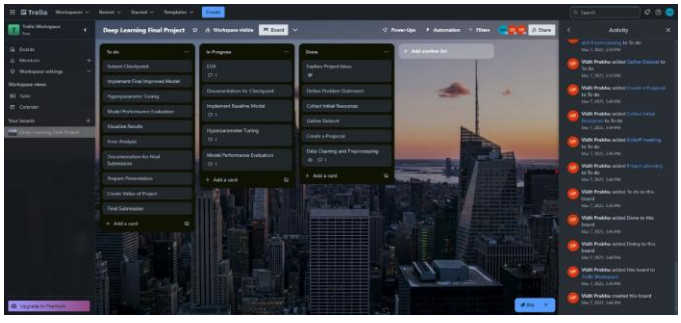
The project employs a Convolutional Autoencoder to learn and reconstruct sign language gestures from the Sign Language MNIST dataset. The input images are first normalized and reshaped for compatibility with convolutional layers. The encoder compresses each image into a lower-dimensional latent space by extracting key spatial features using convolution and pooling layers. The decoder then reconstructs the image from this representation using transposed convolutions. The model is trained using Mean Squared Error (MSE) loss and optimized with the Adam optimizer.

To enhance robustness, a denoising autoencoder setup is used where the model is trained on noisy inputs to improve its ability to reconstruct clean images. Reconstruction error is also leveraged for anomaly detection, where gestures with high error are flagged as outliers. The model's effectiveness is evaluated using both visual inspection of reconstructed outputs and quantitative metrics based on reconstruction loss.

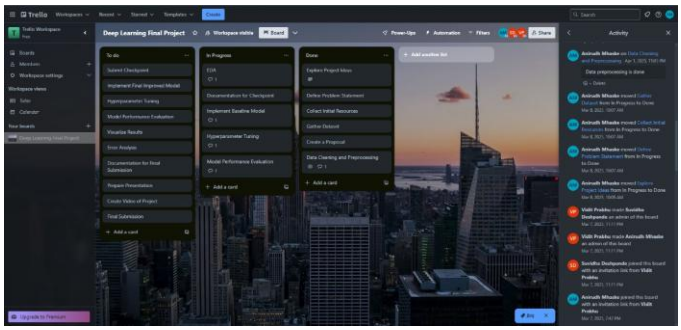
## V. Project Management Tool

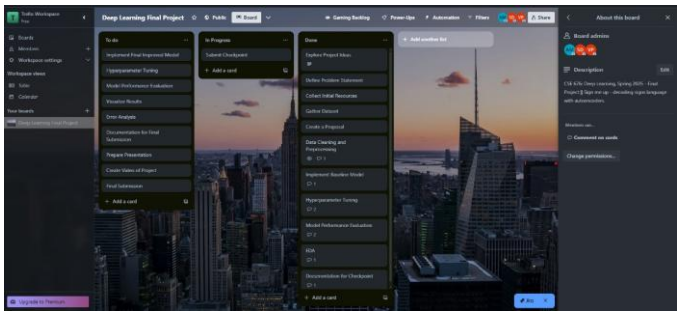
https://trello.com/invite/b/67cb5b2e97d5cddb3c8047d8/ATTIc8fa3c4c0c89d720adf62e76fc3df81cE4D0E9C1/deep-learning-final-project

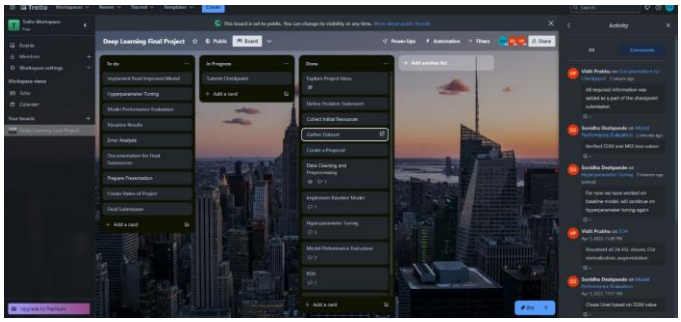


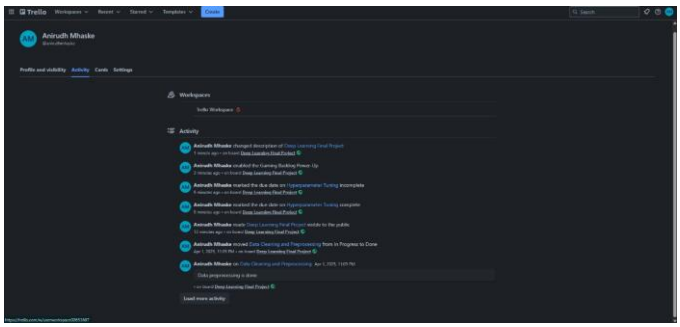


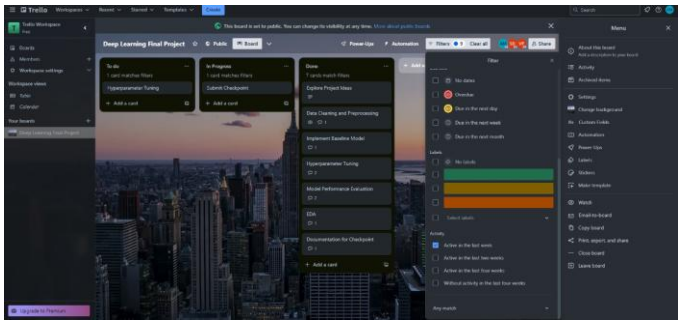
This work is for academic purposes as a part of our course CSE 676 – Deep Learning.

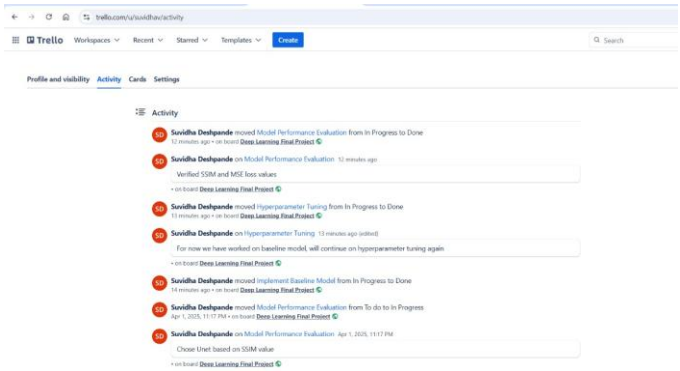


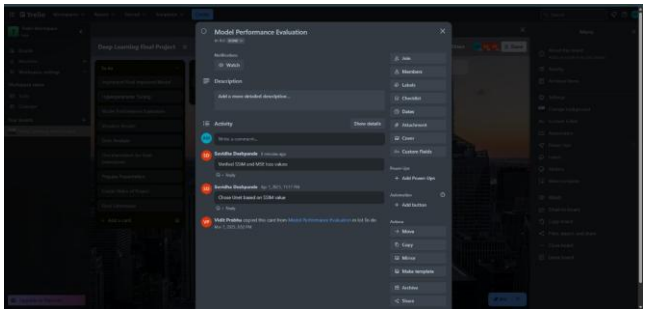


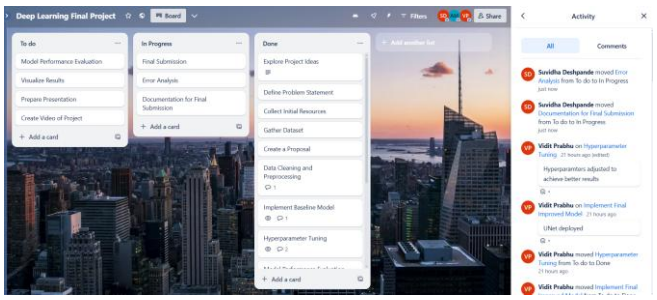


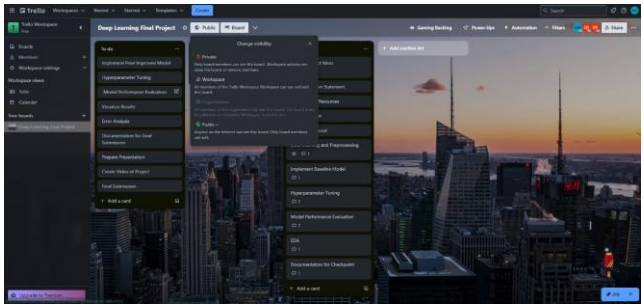


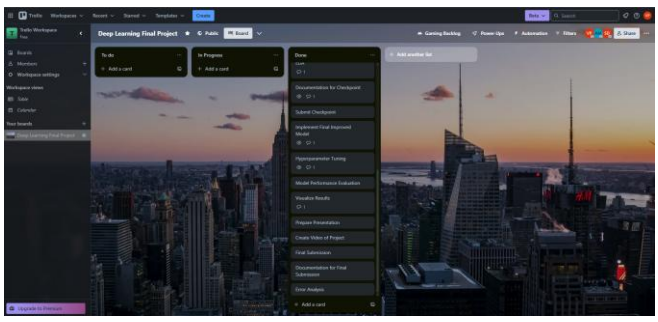












## VI. Visualizations and Results

The dataset used in this study is the Sign Language MNIST dataset, comprising 27,455 grayscale images of hand gestures, each labeled with a corresponding sign (ranging from 0 to 24, excluding 'J' and 'Z' due to dynamic motion). Each image is represented as a flattened vector of 784 pixels (28×28), with an additional label column, resulting in 785 features in total.

A statistical summary of the dataset is presented in Table I, which includes count, mean, standard deviation, and min–max values for each feature across the dataset. The pixel values range from 0 to 255, with an average intensity centered around 145–165 across most pixels, indicating that the images are relatively bright and contain strong gesture features.

DESCRIPTIVE STATISTICS OF DATASET

TABLE I. DESCRIPTIVE STATISTICS OF DATASET (CONTINUED)

| ***Statistic*** | ***Count*** | ***Mean*** | ***Std Dev*** | ***Min*** | ***Max*** |
|---|---|---|---|---|---|
| Samples | 27455 | - | - | - | - |
| Features | 785 | - | - | - | - |
| Pixel 1 | 27455 | 145.42 | 41.36 | 0 | 255 |
| Pixel 784 | 27455 | 159.82 | 64.40 | 0 | 255 |
| Label | 27455 | 12.31 | 7.29 | 0 | 24 |

The below chart shows the number of images for each sign language label (0–24) in the dataset. The distribution is fairly uniform, with most classes containing between 950 and 1300 samples. Label 17 has the most images (1294), whereas label 4 has the fewest (957).

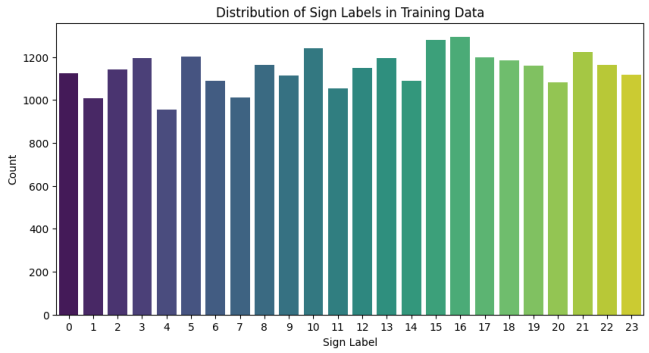



Fig. 1. Distribution of Sign Language Labels.

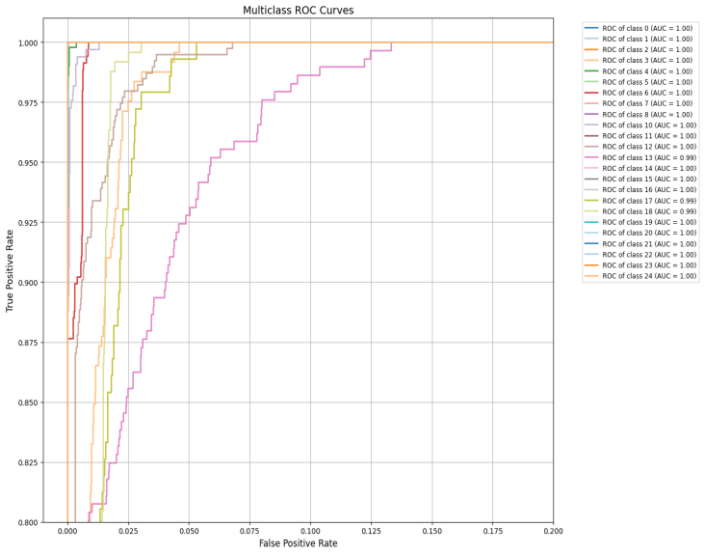
The image showcases example hand gestures from the American Sign Language (ASL) dataset, covering letters A to Y, with the exception of J and Z, which involve motion. Each numbered label (0–24) represents a distinct static gesture used for training and recognizing sign language characters.


Fig. 2. American Sign Language (ASL) Hand Gestures.

The model was trained for 20 epochs, with the training loss decreasing from 1.3502 to 0.053, indicating effective learning over time. Upon evaluation, the final test loss was recorded at 0.1074.The final accuracy , precision , recall and f-1 scores were very promising.The image below shows the results obtained over training our U-NET Autoencoder.

```
Accuracy:  0.9670
Precision: 0.9701
Recall:    0.9670
F1 Score:  0.9667
```

This ROC curve shows how well a multi-class classification model is performing across 25 different classes. What's impressive here is that almost all the classes have an AUC (Area Under the Curve) of 1.00, meaning the model is doing a perfect job for those. A few classes, specifically 13, 17, and 18, have slightly lower AUC scores (around 0.99), but that's still excellent. The graph also zooms into the most relevant part of the ROC space to highlight even small differences in performance. Overall, this model seems to be performing extremely well across the board.




We have deployed our application here https://asl-classifier-dl.streamlit.app/. To deploy the U-Net-based ASL classifier, we followed a modular and reproducible workflow by separating the model logic and deployment interface. Below is a summary of the steps taken:

*1) Model Preparation (model.py)*

Implemented the U-Net Autoencoder architecture using PyTorch.The model consists of reusable components like DoubleConv blocks and a custom UNetClassifier class.The training of the model was done in a separate Jupyter notebook (.ipynb), where the model was trained on an MNIST-style ASL dataset (24 classes excluding 'J' and 'Z').After training, the model weights were saved to a .pth file.In model.py, the trained model was reloaded with the saved weights for inference.

*2) Streamlit Application (app.py)*

Created a user interface using Streamlit, titled "ASL Classifier".Used streamlit.file_uploader() to allow users to upload an image.The uploaded image is passed through a preprocessing pipeline using transforms.Compose, including

grayscale conversion, resizing to 28×28 pixels, and tensor conversion.The preprocessed image is then fed to the model (imported from model.py) to predict the ASL character.

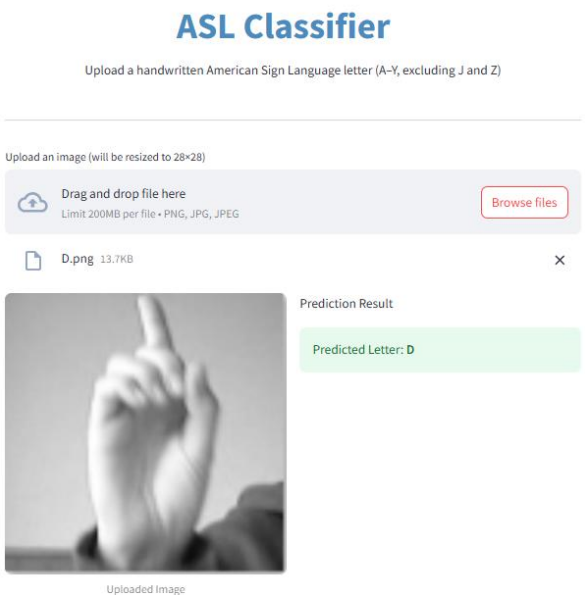
The result is displayed in a user-friendly format on the web UI.

*3) Version Control & Deployment*

Created a new GitHub repository and pushed all necessary files, including app.py, model.py, saved model weights, and requirements.txt.Connected the GitHub repository to Streamlit Cloud.Configured the Streamlit deployment by setting the main file to app.py.

Successfully deployed the app on Streamlit Cloud, enabling real-time ASL image classification from any device.

TABLE II. RESULTS ON THE DEPLOYED APPLICATION



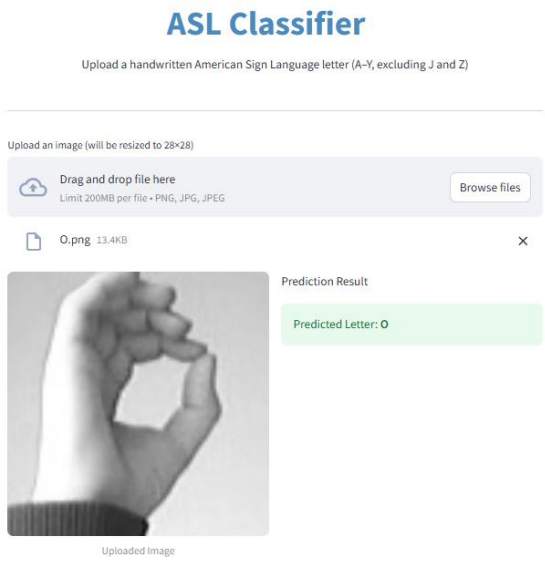




TABLE III. CLASSIFICATION REPORT

| *Label* | *Precision* | *Recall* | *F1 Score* | *Support* |
|---|---|---|---|---|
| 0 | 1 | 0.7341 | 0.8466 | 331 |
| 1 | 1 | 0.9930 | 0.9965 | 432 |
| 2 | 0.9967 | 1 | 0.9983 | 310 |
| 3 | 0.8500 | 0.7632 | 0.8043 | 243 |
| 4 | 1 | 0.9518 | 0.9753 | 498 |
| 5 | 1 | 1 | 1 | 247 |
| 6 | 0.8879 | 0.9339 | 0.9103 | 348 |
| 7 | 0.9498 | 1 | 0.9743 | 436 |
| 8 | 1 | 0.8854 | 0.9392 | 288 |
| 10 | 0.6402 | 1 | 0.7806 | 331 |
| 11 | 1 | 1 | 1 | 496 |
| 12 | 0.9382 | 0.8477 | 0.8906 | 394 |
| 13 | 1 | 0.7216 | 0.8383 | 291 |
| 14 | 0.9959 | 0.9959 | 0.9959 | 246 |
| 15 | 0.9179 | 1 | 0.9572 | 347 |
| 16 | 0.7068 | 1 | 0.8282 | 164 |
| 17 | 0.9589 | 0.4861 | 0.6451 | 144 |
| 18 | 0.6704 | 0.7276 | 0.6978 | 246 |
| 19 | 1 | 0.8346 | 0.9098 | 248 |
| 20 | 0.8260 | 1 | 0.9047 | 266 |
| 21 | 0.9971 | 1 | 0.9985 | 346 |
| 22 | 1 | 1 | 1 | 206 |
| 23 | 1 | 1 | 1 | 267 |
| 24 | 1 | 0.9969 | 0.9984 | 332 |

## VII. CONTRIBUTION SUMMARY

TABLE IV. INDIVIDUAL CONTRIBUTION

| *Team Member* | *Project Part* | *Contribution* |
|---|---|---|
| Anirudh Mhaske | All Parts | 33.33 % |
| Suvidha Deshpande | All Parts | 33.33 % |
| Vidit Prabhu | All Parts | 33.33 % |


## ACKNOWLEDGMENT

We sincerely thank Dr. Alina Vereshchaka for her constant guidance, encouragement, and support throughout this project. We also appreciate the Teaching Assistants for their timely help in resolving doubts and consistently monitoring our progress.


## REFERENCES


[1] Kaggle, “Sign Language MNIST Dataset,” [Online]. Available: https://www.kaggle.com/datasets/datamunge/sign-language-mnist

[2] Y. Xu, T. Yu, Q. Ma, W. Wang, C. Xu, and L. Wang, “One-Stage 3D Whole-Body Mesh Recovery with Component Aware Transformer,” *arXiv preprint arXiv:2104.05670*, 2021. [Online]. Available: https://arxiv.org/pdf/2104.05670

[3] C. Chen, Y. Shen, W. Wang, H. Li, and C. Wang, “Deep Human Pose Estimation: A Survey,” *Computer Vision and Image Understanding*, vol. 210, pp. 103235, 2021.

[4] W. Wang, L. Wang, and H. Li, “Learning Monocular 3D Human Pose Estimation from Multi-view Images,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[5] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *arXiv preprint arXiv:1505.04597*, 2015. [Online]. Available: https://arxiv.org/pdf/1505.04597

[6] [A. Ito Aramendia, “Decoding the U-Net: A Complete Guide,” *Medium*, 2023. [Online]. Available: https://medium.com/@alejandro.itoaramendia/decoding-the-u-net-a-complete-guide-810b1c6d56d8

[7] Scikit-learn Developers, “sklearn.pipeline: Pipeline of Transforms with a Final Estimator,” *Scikit-learn Documentation*, [Online]. Available: https://scikit-learn.org/stable/api/sklearn.pipeline.html

[8] https://docs.streamlit.io/develop